

PROJECT REPORT

CAMPUS LOST & FOUND SYSTEM

A Python-Based Application for Managing Lost and Found Items on Campus

Student Name	Deep Nareshkumar Lad
Registration Number	26BAI10605
Course	CSE1021 – Introduction to Problem Solving and Programming
Program	B.Tech
Faculty / School	AIML
Academic Year	2026–27

1. Introduction

In a college campus, students sometimes lose things such as ID cards, wallets, mobile phones, books, bags, keys and water bottles. Usually, students inform their classmates, post messages in WhatsApp groups or put notices on boards. These methods can work, but information can easily get missed.

The Campus Lost & Found System was developed to provide a simple place where students can report lost and found items. A student can enter details about an item, view existing records, search for an item and check whether a lost item may match a found item.

I selected this project because it is a real problem that can happen in a college and it also gives an opportunity to use the programming concepts learned in the course in one practical application.

2. Problem Statement

There is no single organized system for maintaining information about lost and found belongings on campus. Because information is often shared through different informal channels, a student may not know whether an item has already been found.

The proposed system stores lost and found records in local CSV files. It provides options to add, view, search, update and delete records. It also compares lost and found records using simple matching rules to show possible matches.

3. Objectives

- To provide a simple way to report lost and found items.
- To store item information in an organized form.
- To allow users to search records by name, category and location.
- To provide a simple rule-based method for finding possible matches.
- To allow the status of an item to be updated.
- To demonstrate fundamental Python programming concepts.
- To store data permanently using local CSV files.

4. Scope of the Project

The system is designed for a small college campus and is mainly intended as a console-based application. It can store item name, category, location, date, description, contact number and status.

The current version does not use an online database, login system or mobile application. It is intended as a first-year programming project and therefore uses simple Python and local files instead of advanced frameworks or database systems.

5. Functional Requirements

1. Report Lost Item – A user can enter details of an item that has been lost.
2. Report Found Item – A user can enter details of an item that has been found.

3. View Items – Lost and found records can be displayed.
4. Search Items – Records can be searched by item name, category or location.
5. Find Possible Matches – The system compares lost and found records and gives a simple matching score.
6. Update Status – The status of a record can be changed.
7. Delete Record – A record can be removed after the item is returned or when the record is no longer required.

6. Non-Functional Requirements

- Usability: The program uses a numbered menu so that a beginner can operate it easily.
- Reliability: Records are saved in CSV files and can be read again when the program is restarted.
- Maintainability: The program is divided into separate Python files and functions.
- Error Handling: The program checks invalid choices, empty input, invalid contact numbers and invalid item IDs.
- Resource Efficiency: The application uses local CSV files and does not require a database server.

7. Technologies and Tools Used

Programming Language: Python 3

Storage: CSV files

Development environment: Any Python-supported editor or IDE such as IDLE, VS Code or PyCharm.

The implementation uses variables, input/output, conditional statements, loops, functions, lists, dictionaries, tuples, sets, string operations, file handling and exception handling.

8. System Architecture

The application follows a simple modular structure:

User

|

v

Main Menu

|

+--> Item Functions -----> CSV Files

+--> Search Functions -----> CSV Files

+--> Match Functions -----> Lost + Found Records

+--> Display Functions

+--> Validation Functions

+--> File Functions -----> CSV Files

The main.py file controls the menu. Other modules perform specific tasks. This makes the program easier to understand and modify.

9. Module Description

main.py: Starting point of the application. Displays the main menu and calls functions according to the user's choice.

item_functions.py: Handles adding records, updating status and deleting records.

search_functions.py: Provides searching by name, category and location.

match_functions.py: Compares lost and found records using a simple score.

file_functions.py: Creates CSV files, reads records, saves records and generates item IDs.

validation.py: Checks user input and provides category and contact-number validation.

display_functions.py: Displays records in a readable format.

10. Data Storage Design

Two CSV files are used: lost_items.csv and found_items.csv.

Each record contains:

ID | Name | Category | Location | Date | Description | Contact | Status

A Python dictionary represents one item record. A list is used when several records are loaded into memory. CSV files provide simple permanent storage without requiring a database.

11. Data Structures Used

List: Used for collections such as categories and groups of item records.

Dictionary: Used to represent one item record with keys such as ID, Name, Category and Location.

Tuple: Can be used for fixed field names because they normally do not change while the program is running.

Set: Used in the matching module to compare words in lost and found descriptions and find common words.

These structures are suitable for an introductory Python project and are part of the course syllabus.

12. Matching Algorithm

The matching system is intentionally simple. It is not a machine-learning system. It uses rules to compare a lost item with a found item.

Exact item name match = +3

Similar item name = +2

Same category = +2

Same location = +2

Common description word = +1

If the score reaches 4 or more, the system displays the two records as a possible match.

For example, if a student reports a black wallet lost in the library and another student reports a black wallet found in the library, the records can receive a high score because the name, category and location match.

13. Program Workflow

1. The user starts the program.
2. The main menu is displayed.
3. The user selects an operation.
4. If the user reports an item, details are collected and validated.
5. The record is stored in the appropriate CSV file.
6. The user can later view or search records.
7. The matching option compares lost and found records.
8. The user can update or delete a record.
9. The program continues until the user selects Exit.

14. Algorithms and Python Concepts Demonstrated

- Input and output using `input()` and `print()`.
- Conditional statements using `if`, `elif` and `else`.
- Iteration using `for` and `while` loops.
- Functions with parameters and return values.
- Lists for storing multiple values.
- Dictionaries for representing item records.
- Tuples for fixed collections of values.
- Sets and set intersection for matching description words.

- String functions such as lower(), strip() and split().
- File handling using CSV files.
- Exception handling using try and except.
- Searching through records using loops and conditions.
- Counting records using length and list operations.

15. Error Handling and Validation

The system checks common input errors. Empty fields are not accepted for important information. The contact number is checked to make sure that it contains 10 digits. Menu choices are checked before performing an operation.

When an item ID is entered, exception handling prevents the program from stopping if the user enters something that cannot be converted into an integer.

16. Testing Approach

Test 1: Add a lost item with valid details. Expected: the record is saved in lost_items.csv.

Test 2: Add a found item with valid details. Expected: the record is saved in found_items.csv.

Test 3: Search for an existing item. Expected: matching records are displayed.

Test 4: Search for an item that does not exist. Expected: no matching record message is displayed.

Test 5: Enter an invalid menu option. Expected: an invalid-choice message is displayed.

Test 6: Enter an invalid contact number. Expected: the program asks for a valid 10-digit number.

Test 7: Run matching with similar lost and found records. Expected: a possible match and score are displayed.

Test 8: Update and delete an existing record. Expected: the CSV file is changed accordingly.

17. Challenges Faced

One challenge was deciding how to store item information. Storing many separate variables would make the program difficult to manage, so dictionaries were used for individual records and lists for multiple records.

Another challenge was creating the matching feature without using advanced techniques. A simple scoring method was selected because it can be understood using basic programming concepts.

File handling was also important because records should not disappear when the program closes. CSV files provided a simple solution.

18. Design Decisions and Rationale

The project was deliberately kept as a console application because the main aim was to demonstrate programming fundamentals.

CSV storage was selected instead of a database because the expected amount of data for a small demonstration project is limited and CSV is easier to understand at an introductory level.

The program was divided into modules because modular implementation makes debugging and maintenance easier.

19. Results

The completed system provides a basic working environment for managing campus lost and found records. A user can report items, view them, search records, compare lost and found items, update their status and delete records.

The matching mechanism adds a useful feature because users do not have to manually compare every lost and found record.

20. Future Enhancements

The project can later be improved by adding a graphical interface, a proper database, student login, image upload, email notifications, better filtering, administrator controls, a web/mobile version and more advanced matching.

These features were not included in the current version because the main purpose is to demonstrate the fundamental programming concepts covered in the first-year course.

21. Learnings and Key Takeaways

While developing this project, I learned how separate Python concepts can be combined to solve a real problem. Functions helped divide the program into smaller tasks. Lists and dictionaries made it easier to store records. Sets were useful for comparing description words. File handling helped make the data permanent.

I also learned that a program does not have to use complicated technology to solve a useful problem. A simple console application can still have a clear workflow and practical features.

22. Conclusion

The Campus Lost & Found System provides a simple and organized alternative to informal lost-and-found communication on a college campus. The application allows users to store, search and manage lost and found records and provides a basic method for identifying possible matches.

The project demonstrates practical use of Python concepts covered in the Introduction to Problem Solving and Programming course. Since the program is modular and uses local files, it can also be extended later into a larger application.

23. References

1. CSE1021 Introduction to Problem Solving and Programming – course syllabus.

2. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
3. Python documentation and basic Python programming concepts used during implementation.

24. Screenshots / Results to Add Before Submission

Screenshot 1 – Main Menu

[Insert your actual screenshot here]

Screenshot 2 – Reporting a Lost Item

[Insert your actual screenshot here]

Screenshot 3 – Viewing Lost/Found Records

[Insert your actual screenshot here]

Screenshot 4 – Searching for an Item

[Insert your actual screenshot here]

Screenshot 5 – Possible Match Result

[Insert your actual screenshot here]

Screenshot 6 – Updating or Deleting a Record

[Insert your actual screenshot here]

25. Appendix – Project Files

main.py

item_functions.py

search_functions.py

match_functions.py

file_functions.py

validation.py

display_functions.py

lost_items.csv

found_items.csv